EcoCanje — Expediente técnico (D01–D10)

Práctica individual Kubernetes: desarrollo y producción · Infraestructura de Servicios, ITM

DaBriNan

Noviembre de 2026 · versión del código v1.0.0

Índice del expediente

Documento	Contenido	Archivos principales del repositorio
D01	Ficha, reglas de negocio, requerimientos funcionales y no funcionales con criterios de aceptación	backend/app, tests/
D02	Investigación de Kubernetes y diez decisiones de arquitectura	infra/, scripts/crear-clusters.sh
D03	Diagramas de aplicación, secuencia, implementación y comunicación; inventario y capacidad	docs/diagramas/*.mmd
D04	Modelos y diccionarios de PostgreSQL y MongoDB, migraciones, consistencia y conexiones	backend/database/migrations, PrepararMongo.php
D05	Manual de instalación desde cero y verificación	scripts/bootstrap.sh, README.md
D06	Seguridad: controles, RBAC, cuentas de BD,	infra/base/rbac.yaml, redes.yaml, infra/firewall/

Documento	Contenido	Archivos principales del repositorio
	flujos, firewall, secretos, vulnerabilidades, riesgos	
D07	Publicación, promoción, actualización, rollback y accesos	scripts/actualizacion.sh, promover-frontend.sh, .github/workflows/ci.yml
D08	Operación, observabilidad, alertas y manual de incidentes	infra/observabilidad/, scripts/prom.sh, loki.sh
D09	Respaldos, restauración y continuidad con RPO y RTO medidos	infra/base/respaldos.yaml, scripts/restaurar.sh
D10	Plan de pruebas, matriz E01–E14, hallazgos, conclusiones y limitaciones	docs/evidencias/INDEX.md

Las evidencias citadas (E01–E14, OBS, SUP, DEP, SEC, INC) están en docs/evidencias/, con su salida completa y su fila en INDEX.md.

D01. Ficha del proyecto y requerimientos

Ficha

Campo	Valor
Proyecto	EcoCanje: centro de acopio y reciclaje con canje de puntos
Materia	Infraestructura de Servicios — práctica individual de Kubernetes (ITM)
Repositorio	https://github.com/DaBriNan/EcoCanje (rama dev; versión entregada etiquetada v1.0.0)
Problema	Un centro de acopio necesita registrar el material que entregan los ciudadanos, pesarlo y validarlo,

Campo	Valor
	acreditar puntos según una tarifa vigente por material y permitir canjear esos puntos por recompensas, con trazabilidad de cada movimiento.
Usuarios	**Ciudadano**: consulta tarifas, su saldo, sus entregas e historial; solicita canjes. **Operador**: registra y procesa entregas, administra el catálogo (materiales, tarifas, fichas, recompensas), aprueba y entrega canjes, consulta el reporte mensual.
Alcance técnico	SPA React, API Laravel, PostgreSQL (fuente de verdad) y MongoDB (libro de movimientos y fichas) en dos clústeres k3s independientes (dev y prod) sobre k3d, con observabilidad, respaldos, seguridad y automatización.

Reglas de negocio principales

Regla	Implementación
Una entrega pasa por registrada → pesada → validada → acreditada; puede rechazarse con motivo desde los tres primeros estados. Cualquier otra transición devuelve 409.	EntregaController::paso() con bloqueo de fila (lockForUpdate)
Los puntos se calculan con la tarifa vigente **al registrar** cada detalle (peso × puntos/kg, redondeado). La tarifa vigente es la más reciente con vigente_desde <= ahora; empate en el mismo segundo: la de mayor id.	EntregaController::store, acreditar
Tope diario de 500 puntos acreditados por ciudadano; los puntos vencen a los 12 meses.	PuntosService::TOPE_DIARIO, VIGENCIA_MESES
Un canje consume puntos de los lotes más antiguos primero (FIFO);	PuntosService::consumir, CanjeController::cancelar

Regla	Implementación
cancelarlo devuelve los puntos a sus lotes y el stock.	
Un canje pasa por solicitado → aprobado → entregado; puede cancelarse desde los dos primeros.	CanjeController::paso()
Materiales y recompensas se dan de baja lógicamente (activo/activa = false) para conservar el historial de entregas y canjes.	MaterialController::destroy, RecompensaController::destroy
Cada cambio de estado de negocio registra un evento en el *outbox* en la misma transacción; el worker lo publica en MongoDB de forma idempotente.	PuntosService::evento, PublicarOutbox

Requerimientos funcionales

ID	Requerimiento (PDF §2)	Criterio de aceptación	Verificación
RF01	Interfaz navegable y adaptable con formularios, errores y confirmaciones	Ambas vistas operan en escritorio y en celular (390 px); cada operación muestra confirmación o error	tests/interfaz/e03.mjs (capturas), E03
RF02	Backend con API y reglas de dominio; el navegador no accede a las bases	Las bases no tienen Service expuesto ni ruta de Ingress; el frontend no alcanza las bases (NetworkPolicy)	tests/netpol-frontend.sh, E08
RF03	CRUD de tres entidades relacionadas	Materiales (con tarifas y ficha), recompensas y	tests/catalogo.sh, tests/flujo.sh, E03

ID	Requerimiento (PDF §2)	Criterio de aceptación	Verificación
		entregas (con detalles): alta, consulta, modificación y baja lógica	
RF04	Autenticación y dos roles con autorización en servidor	Ciudadano recibe 403 en rutas de operador; sin token 401 aunque falte Accept: application/json	tests/roles.sh, E09
RF05	Validación de tipos, obligatorios, rangos, unicidad y relaciones, en cliente y servidor	El navegador bloquea datos fuera de rango; la API devuelve 422 por campo aunque se evite la interfaz	tests/catalogo.sh, e03.mjs, E03
RF06	Búsqueda, paginación y un reporte útil	Entregas filtrables por estado y ciudadano, paginadas de 15 en 15; historial paginado; reporte mensual por material	tests/catalogo.sh, e03.mjs
RF07	Flujo propio con estados y transiciones válidas	Entregas y canjes con las transiciones de la tabla anterior; transiciones inválidas → 409	tests/flujo.sh (22 verificaciones)
RF08	Semilla reproducible de al menos 100 registros sin datos	db:seed crea operador, ciudadanos ficticios, catálogo	Job migraciones, scripts/estado-datos.sh

ID	Requerimiento (PDF §2)	Criterio de aceptación	Verificación
	reales	y más de 100 entregas	
RF09	Uso real de la base no relacional	Historial de movimientos y fichas técnicas de material se leen y escriben en MongoDB con validación de esquema e índices	tests/catalogo.sh, tests/correlacion.sh
RF10	Fuente de verdad y recuperación entre motores sin duplicados	PostgreSQL es la fuente de verdad; con MongoDB caído las escrituras siguen y el outbox reconcilia sin duplicados	scripts/fallos.sh prod mongo, E04, E11

Requerimientos no funcionales

ID	Requerimiento	Criterio de aceptación	Resultado medido
RNF01	Dos ambientes independientes	Clústeres, contextos, credenciales, datos y volúmenes separados; prod con 1 control plane y 2 workers	tests/ independencia.sh, E01, E02
RNF02	Publicación HTTPS con nombres distintos	dev.ecocanje.test:8444 y prod.ecocanje.test:8445 con TLS de la CA local	E02
RNF03	Rendimiento bajo	p95 < 1 s y errores	**p95 0.99 s, 0 %**

ID	Requerimiento	Criterio de aceptación	Resultado medido
	carga	< 1 % con 50 usuarios virtuales en prod	**errores, 124 rps** (E06, con HPA); sin HPA: p95 12.6 s, 13 %
RNF04	Escalado horizontal	HPA 2–6 crece por encima de 2 con carga y regresa al mínimo	2 → 6 → 2 en 333 s (E06)
RNF05	Disponibilidad ante fallos voluntarios	Borrado de Pod y drain de worker sin solicitudes fallidas	0 errores en ambos casos (E11)
RNF06	Actualización sin corte y rollback	Rolling update, rollback y nueva promoción sin errores en la sonda	0 errores en las tres fases (E12)
RNF07	Observabilidad	Métricas de nodos, Kubernetes, aplicación y ambas bases; logs centralizados con correlación; 8 alertas, 2 provocadas	E10, OBS04–OBS07
RNF08	Recuperación	RPO objetivo 24 h y RTO objetivo 30 min, medidos	ver D09
RNF09	Seguridad	Pod Security restricted, NetworkPolicy de denegación inicial, RBAC por función, secretos	E08, E09, SUP03, D06

ID	Requerimiento	Criterio de aceptación	Resultado medido
		fuera de Git, imágenes por digest, análisis de vulnerabilidades	
RNF10	Reconstrucción reproducible	Dev y prod desde cero con un solo script, sin pasos tácitos	759 s, 671 s y una tercera corrida con k3s v1.36.5 (E14, ver D05)

D02. Investigación de Kubernetes y decisiones de arquitectura

Conceptos aplicados

- **Clúster, nodo, control plane y worker.** k3s empaqueta API server, scheduler, controller-manager y etcd/kine en un solo binario. En prod el control plane corre en k3d-prod-server-0; las cargas de la aplicación se fijan a los workers con afinidad de nodo (node-role.kubernetes.io/control-plane DoesNotExist).
- **Deployment, StatefulSet, Job y CronJob.** Backend, frontend y worker del outbox son Deployments sin estado; las bases son StatefulSets con un PVC por réplica; las migraciones son un Job único por despliegue y los respaldos, CronJobs.
- **Service frente a publicación.** Un Service ClusterIP (backend, frontend, postgres, mongo) solo reparte conexiones entre endpoints **dentro** del clúster. Lo que publica tráfico fuera es el controlador de entrada (Traefik) detrás del balanceador de k3d (k3d-*-serverlb, un contenedor nginx en el host que reenvía los puertos publicados al Service LoadBalancer de Traefik, respaldado por ServiceLB/klipper-lb de k3s). Un recurso Ingress sin su controlador no publica nada.
- **IP externa en laboratorio.** No hay IP pública: k3d publica los puertos del balanceador en 127.0.0.1 del host y /etc/hosts resuelve dev.ecocanje.test y prod.ecocanje.test a esa dirección.
- **HPA y recursos.** El HPA calcula el porcentaje de CPU sobre los *requests* (backend: 100m + 20m por Pod); crea Pods, no máquinas: con 6 réplicas se alcanzó el límite de CPU de los Pods (D10, E06).

- **PDB.** Regula desalojos voluntarios (drain), no fallos imprevistos: no evita la caída de un nodo.

Decisiones

#	Decisión	Alternativa descartada	Motivo	Costo	Limitación
ADR-01	**Distribución: k3s v1.36.5 en k3d** (contenedores Docker como nodos), dos clústeres en el mismo host	minikube; kubeadm en VMs	k3d crea clústeres multinodo reales (1+2) en segundos y reproducibles por script; k3s es una distribución certificada y ligera	0 (software libre); comparte CPU/RAM del host	Los "nodos" comparten kernel y disco: la falla del host tumba ambos ambientes. Tras reiniciar o suspender el host, Docker puede intercambiar IP de agentes (incidente E11)
ADR-02	**CNI: flannel (VXLAN)** + **kube-router** para NetworkPolicy, ambos integrados en k3s	Calico; Cilium	Hace cumplir NetworkPolicy sin componentes extra; probado A/B (E08)	Ninguno adicional	kube-router rechaza con REJECT y programa reglas con un breve retraso para Pods nuevos (observado en E10: un

#	Decisión	Alternativa descartada	Motivo	Costo	Limitación
					Job falló por *Connection refused* a los 2 s)
ADR-03	**Entrada: Ingress con Traefik 3.7** (incluido en k3s), TLS con certificado de la CA local (mkcert)	Gateway API; ingress-nginx	Traefik viene mantenido y versionado con k3s (sube con la versión de k3s: 3.6.13 → 3.7.13 en v1.36.5); Ingress basta para dos rutas (/ y /api)	Ninguno	Gateway API daría roles y rutas más expresivas; el certificado solo es confiable en el host con la CA instalada
ADR-04	**Persistencia: local-path-provisioner** (StorageClass local-path)	Longhorn; NFS	Incluido en k3s, sin dependencias	Ninguno	El volumen queda atado al nodo: con el drain de un worker su base no puede moverse y queda detenida hasta el uncordon (E11). En producción

#	Decisión	Alternativa descartada	Motivo	Costo	Limitación
					real: almacenamiento replicado o base con réplicas
ADR-05	**Motores de datos: PostgreSQL (fuente de verdad)** + **MongoDB** (movimientos y fichas), sincronizados con **outbox idempotente**	Escritura doble en ambos motores; solo PostgreSQL con JSONB	Las operaciones de puntos requieren transacciones y llaves foráneas; el historial y las fichas son documentos flexibles. El outbox evita suponer atomicidad entre motores	Un worker adicional y latencia de segundos en el historial	Una instancia por motor (sin réplicas): cada base es punto único de fallo
ADR-06	**Observabilidad: kube-prometheus-stack** + **exportadores de ambas bases; Loki** + **Alloy para logs**	ELK/EFK; Promtail	Pila estándar con CRD (PodMonitor, PrometheusRule) y panel como código; Alloy reemplaza a Promtail	~1 core y ~1.5 GiB en prod	Monitoreo dentro del mismo clúster: si caen los workers, cae también (incidente del 5 nov)

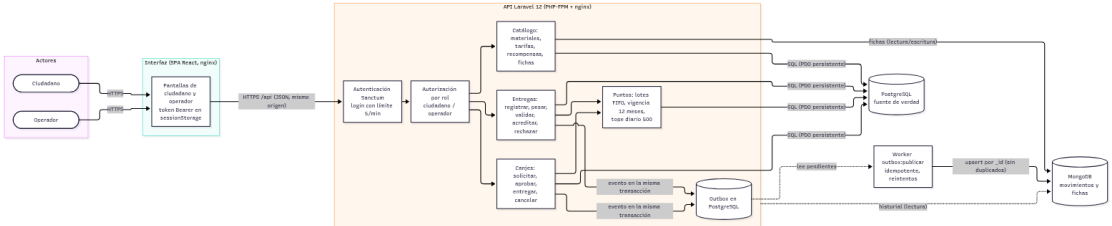
#	Decisión	Alternativa descartada	Motivo	Costo	Limitación
			(obsoleto) y lee por la API sin privilegios		
ADR-07	**Configuración declarativa: Kustomize** (base + overlays dev/prod) y Helm solo para charts de terceros, con versiones fijadas	Charts propios de Helm	Diferencias entre ambientes visibles en un parche pequeño; sin plantillas	Ninguno	Los valores sensibles nunca van en Kustomize: los crean scripts en cada clúster
ADR-08	**SPA servida en el mismo origen** (/ frontend, /api backend) con token Bearer en sessionStorage	Cookies de sesión con CSRF; dominios separados con CORS	Sin CORS ni CSRF; el token se pierde al cerrar la pestaña y expira en 8 h	Ninguno	Un XSS podría leer el token; se mitiga con CSP estricta (script-src 'self')
ADR-09	**Conexiones persistentes a PostgreSQL** (PDO) por	Una conexión por solicitud; PgBouncer	La autenticación scram por solicitud saturó la CPU de	Hasta 6 réplicas × 6 procesos = 36 conexiones de 100	Tras reiniciar PostgreSQL las conexiones rotas se descartan

#	Decisión	Alternativa descartada	Motivo	Costo	Limitación
	proceso PHP-FPM		PostgreSQL (E06, 0.1.1)		al primer error (el worker lo hace explícitamente)
ADR-10	**Publicación solo en 127.0.0.1** de la API de Kubernetes y la entrada	Firewall del host como único control	Docker inserta sus reglas antes que firewalld: publicar en 0.0.0.0 dejaba la API alcanzable desde la LAN y la VPN (E08)	Ninguno	Acceso solo desde el propio host; el firewall de host queda documentado (D06)

D03. Diagramas, inventario y presupuesto de capacidad

Los diagramas son editables en docs/diagramas/ (Mermaid, texto plano); las imágenes se generan con scripts/informe.sh.

Arquitectura de la aplicación

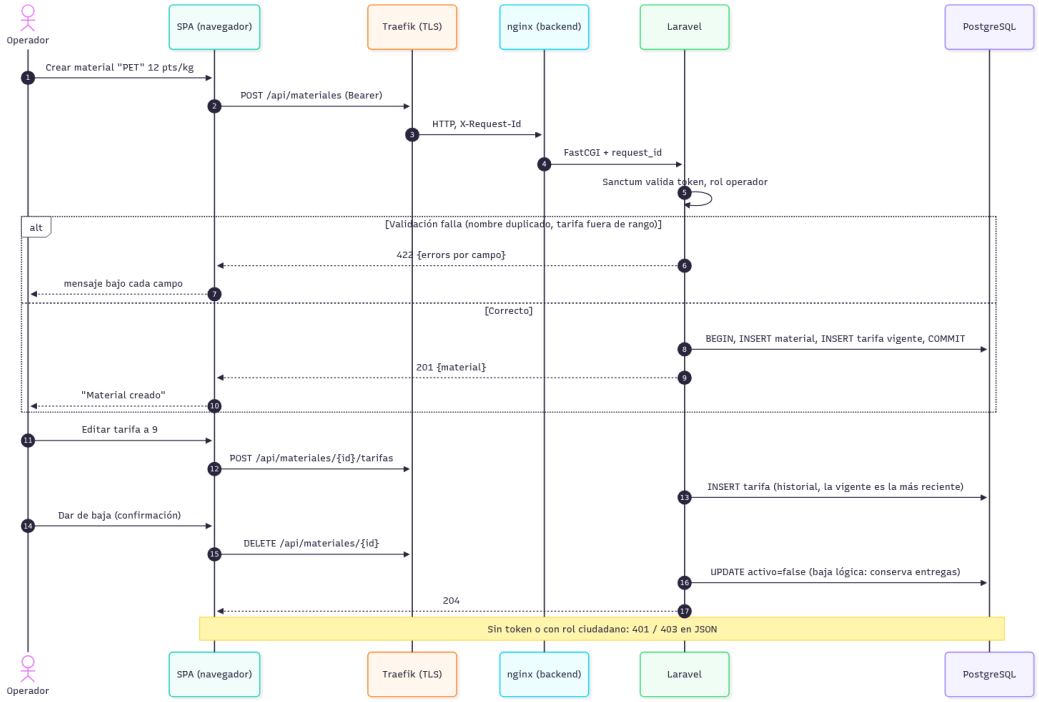


Arquitectura de la aplicación

Recorrido de referencia (SPA en el mismo origen): navegador → 127.0.0.1:8445 (balanceador de k3d) → Traefik (TLS, Ingress) → / Service frontend → réplicas nginx con la SPA; las llamadas de la SPA van a /api → Service backend → réplicas nginx + PHP-FPM → Services internos postgres y mongo. El navegador nunca llega a las bases: no tienen ruta de Ingress, su Service es ClusterIP y las NetworkPolicy solo admiten a los clientes etiquetados (E08). Se eligió SPA porque toda la lógica y la autorización ya están en la API; el frontend es estático y escala sin estado.

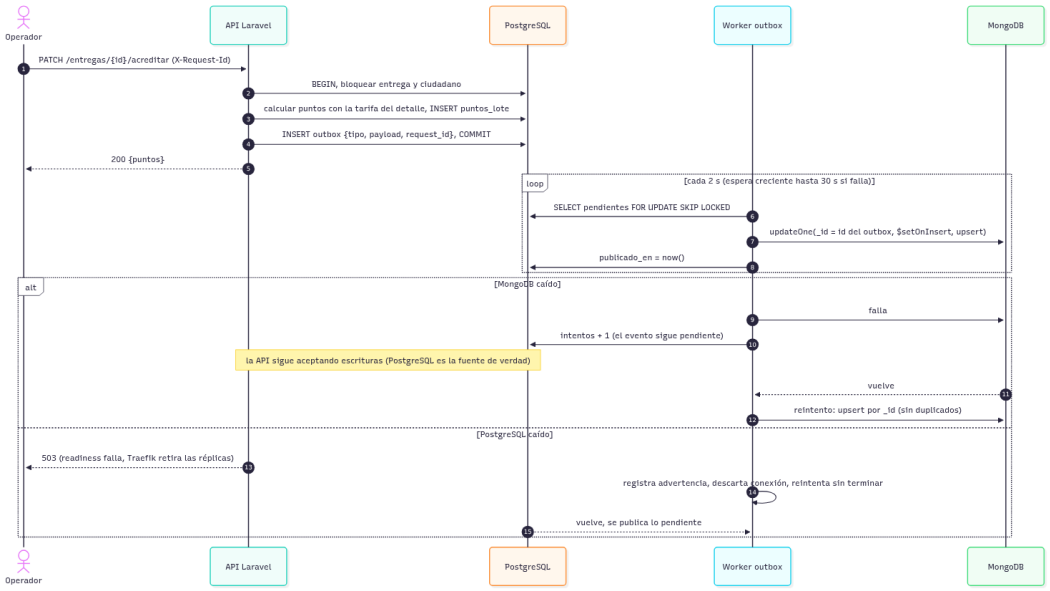
Secuencias

CRUD de materiales (validación, alta, cambio de tarifa y baja lógica):



Secuencia de un CRUD

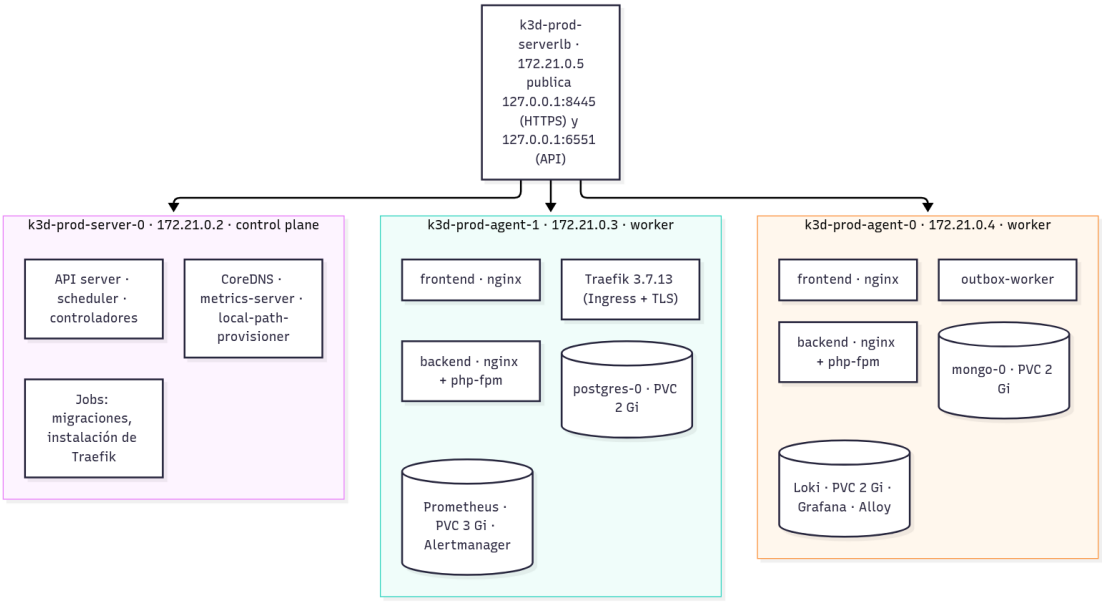
Flujo que usa ambos motores, con errores y recuperación:



Secuencia con ambos motores

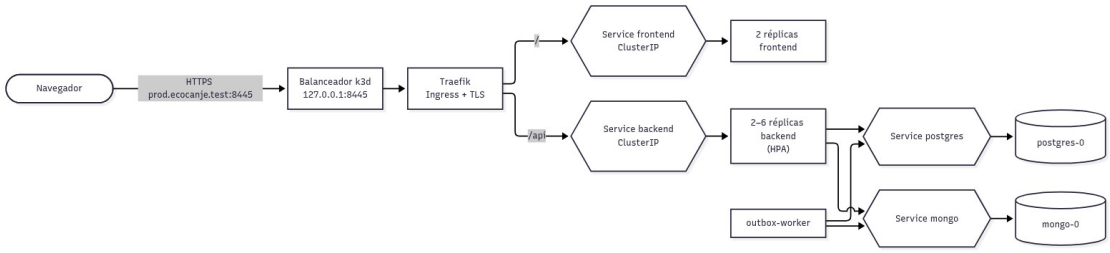
Implementación por ambiente

Ubicación de cada componente por nodo (observada el 2026-11-06; los componentes de una réplica los ubica el scheduler y pueden cambiar al recrear el clúster, salvo las bases, atadas al nodo de su volumen):

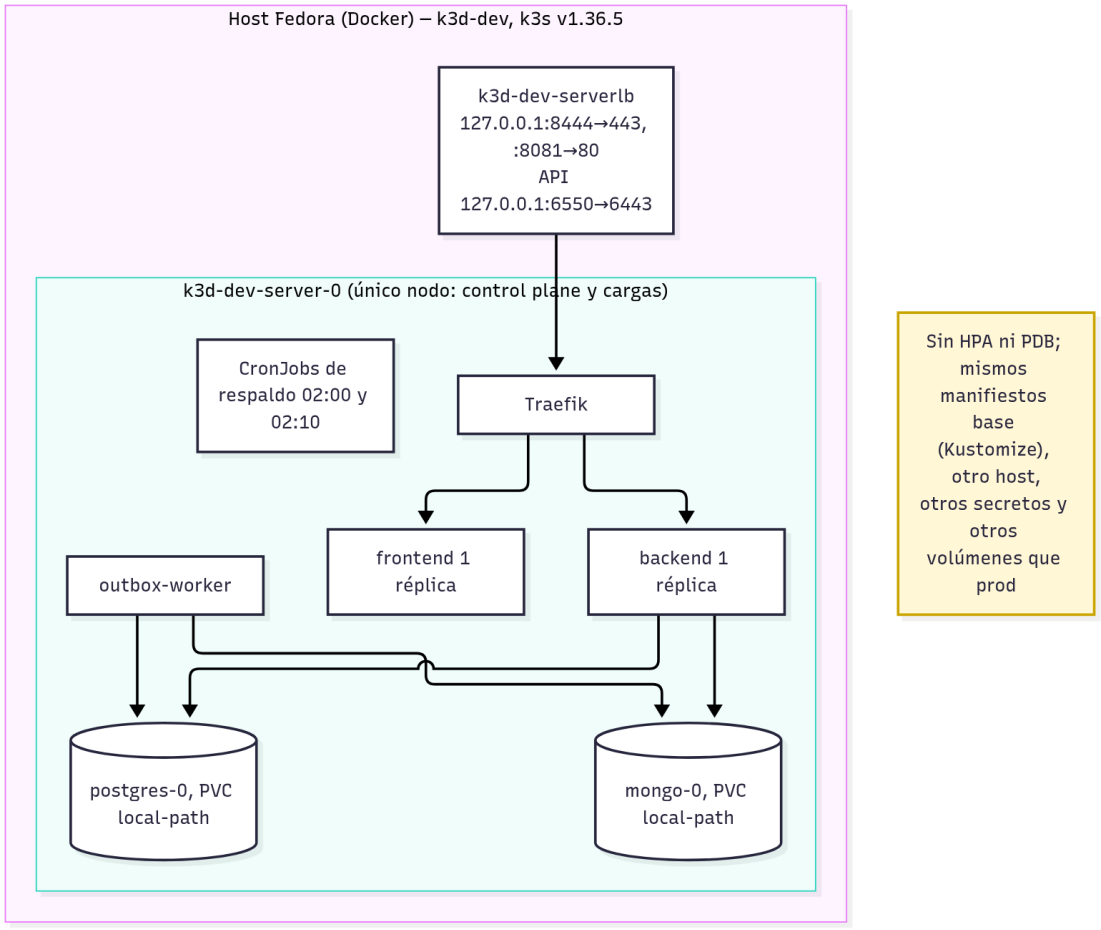


Implementación de producción por nodo

Flujo lógico en producción: el Ingress publica, los Services reparten entre réplicas dentro del clúster:



Flujo lógico de producción

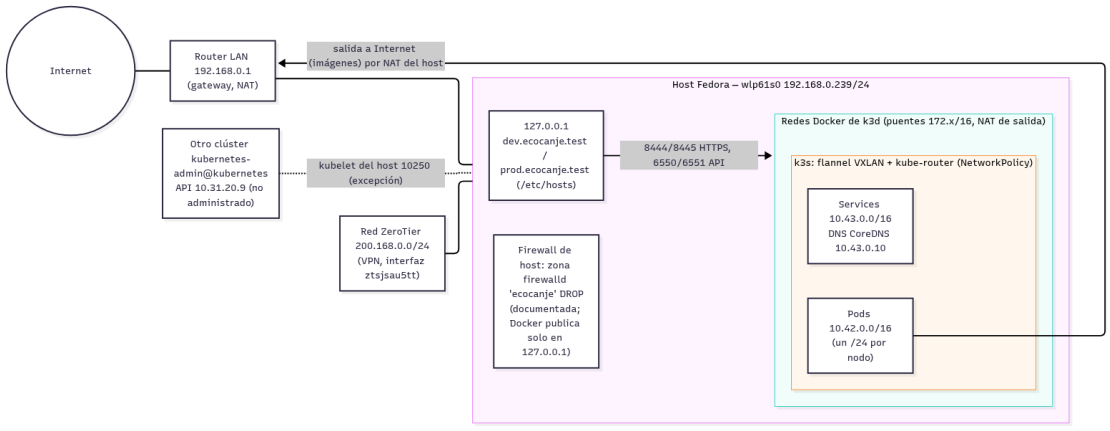


Implementación de desarrollo

Objeto	dev	prod
Nodos	1 (server, también corre cargas)	1 server (control plane) + 2 workers; cargas de la app y Traefik en workers
backend / frontend	1 / 1 réplica	2–6 (HPA, CPU 60 %) / 2,

Objeto	dev	prod
		una por worker (topologySpreadConstraints, nodeTaintsPolicy: Honor, matchLabelKeys)
outbox-worker	1	1
Bases	StatefulSets postgres-0 y mongo-0, PVC local-path 2 Gi c/u	igual, en workers distintos
Respaldos	CronJobs 02:00 y 02:10, PVC respaldos	igual; exportación cifrada fuera del clúster
PDB	—	backend y frontend minAvailable: 1
Observabilidad	—	Prometheus (PVC 3 Gi), Alertmanager, Grafana, kube-state-metrics, node-exporter, exportadores de ambas bases, Loki (PVC 2 Gi), Alloy

Comunicación



Arquitectura de comunicación

Elemento	dev	prod	Notas
Red Docker del clúster	172.20.0.0/16 (gw .1)	172.21.0.0/16 (gw .1)	puentes de Docker, NAT de salida del host

Elemento	dev	prod	Notas
Nodos	server-0 172.20.0.2, serverlb .3	server-0 172.21.0.2, agent-1 .3, agent-0 .4, serverlb .5	las IP de los agentes pueden cambiar al reiniciar el host (incidente E11)
CIDR de Pods	10.42.0.0/24	10.42.0.0/24 (server), 10.42.1.0/24 (agent-1), 10.42.2.0/24 (agent-0)	flannel VXLAN
CIDR de Services	10.43.0.0/16	10.43.0.0/16	DNS CoreDNS 10.43.0.10
Publicación	127.0.0.1:8444 (HTTPS), :8081, API :6550	127.0.0.1:8445 (HTTPS), :8082, API :6551	solo loopback (E08 tests/exposicion.sh)
Nombre	dev.ecocanje.test	prod.ecocanje.test	/etc/hosts del host

Rangos del host: LAN 192.168.0.0/24 (wlp61s0, gateway 192.168.0.1), VPN ZeroTier 200.168.0.0/24 (ztsjsau5tt) y la red del otro clúster 10.31.0.0/16. No se solapan con los rangos de Pods (10.42/16), Services (10.43/16) ni las redes de Docker (172.17–172.21/16). Los clústeres son independientes: mismos rangos internos, pero redes Docker, nodos, credenciales y volúmenes distintos (tests/independencia.sh).

Inventario

Recurso	Valor
Equipo	Laptop, Intel Core i7-8650U a 1.90 GHz, x86_64, 4 núcleos / 8 hilos
Memoria	23 GiB
Disco	NVMe, partición raíz 200 GB (139 GB libres)
Sistema	Fedora Linux 44 Workstation, kernel 7.2.15
Contenedores	Docker 29.9.3; k3d con k3s v1.36.5
IP	192.168.0.239/24 (Wi-Fi), 200.168.0.179/24 (ZeroTier)

Aislamiento y capacidad compartida. Cada nodo de k3d declara como asignables los 8 CPU y 23 GiB del host completo; los tres nodos de prod y el de dev compiten por el mismo procesador, memoria y disco. Kubernetes no lo sabe: el presupuesto real es el del host. Una falla del host detiene ambos ambientes.

Presupuesto de capacidad (medido el 2026-11-06)

Requests y límites declarados en reposo:

Ambiente	Pods	Requests CPU	Requests memoria	Límites CPU	Límites memoria
dev	10	0.58	876 Mi	3.05	2.6 Gi
prod: ecocanje	9	0.53	952 Mi	4.35	3.1 Gi
prod: monitoring	7	0.27	1074 Mi	2.70	2.7 Gi
prod: monitoring-nodos	3	0.03	72 Mi	0.60	192 Mi
prod: kube-system	7	0.20	140 Mi	—	170 Mi
prod total	26	1.03	2.2 Gi	7.65	6.2 Gi

Consumo real de prod en reposo: ~1.3 cores y ~3.2 GiB entre los tres nodos. Cuota del namespace ecocanje: requests 4 CPU / 6 Gi, límites 8 CPU / 10 Gi, 40 Pods, 6 PVC, 20 Gi de almacenamiento.

Con el HPA al máximo (6 réplicas de backend): +4 × (120m requests, 700m límites) → límites de CPU de ecocanje 7.15 de 8. Bajo la carga de E06 el host llegó a 63 % de CPU y el backend a 2.97 cores con 89 % de throttling: el siguiente límite es la CPU de los Pods y del propio host, no la cuota. **Restricción operativa:** si el HPA está en 6, el Job de migraciones (límite 1 CPU) excedería la cuota de límites (8.15 > 8) y no se crearía; no desplegar con el HPA al máximo, o subir la cuota.

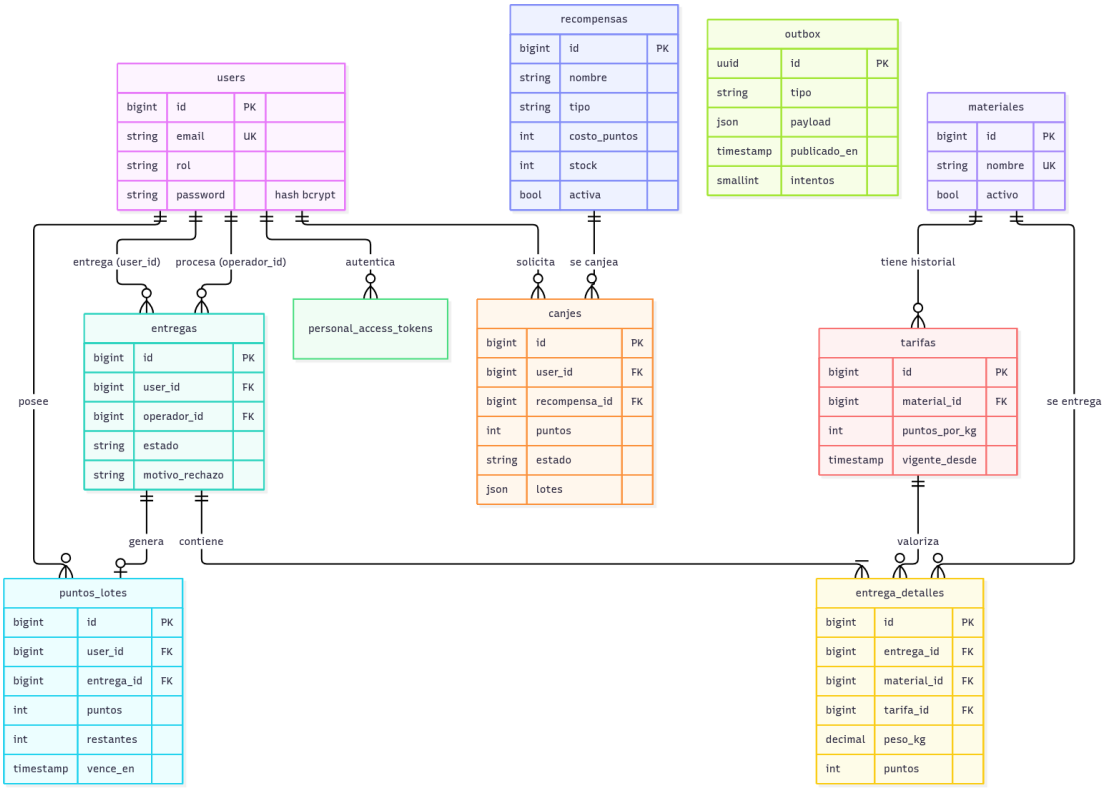
Cuando ya no caben réplicas. El HPA crea Pods, no máquinas: con 6 réplicas se alcanza maxReplicas; si la cuota o la memoria del host se agotan antes, los Pods nuevos quedan en Pending (visibles en el panel y en kube_pod_status_phase). El crecimiento real requeriría más hosts o un host mayor.

Puntos únicos de fallo

Componente	Consecuencia	Mitigación posible
Host físico	ambos ambientes caen	segundo host o nube
Control plane único (server-0)	sin cambios ni reprogramación; las cargas siguen	k3s con 3 servers (etcd embebido)
Balanceador de k3d (serverlb)	sin entrada externa	balanceador redundante
PostgreSQL y MongoDB (1 réplica, volumen local)	sin servicio (PG) o historial retrasado (Mongo)	réplicas del motor y almacenamiento replicado
Monitoreo en los mismos workers	ciego si caen los workers	Prometheus fuera del clúster o en otro clúster

D04. Modelos y diccionarios de datos, migraciones y consistencia

PostgreSQL (fuente de verdad)



diagram

Diccionario

Tabla	Columna	Tipo	Restricción / índice	Significado
users	rol	varchar(20)	default ciudadano	ciudadano u operador; define la autorización en la API
materiales	nombre	varchar(80)	UNIQUE	Material reciclable; activo=false es baja lógica

Tabla	Columna	Tipo	Restricción / índice	Significado
tarifas	material_id, vigente_desde	FK, timestamp	índice (material_id, vigente_desde)	Historial de precios; vigente = la más reciente <= ahora (desempate por id)
entregas	estado	varchar(20)	índice (user_id, estado)	registrada, pesada, validada, acreditada, rechazada
entrega_detalles	tarifa_id	FK a tarifas		Tarifa congelada al registrar: un cambio posterior no altera la entrega
entrega_detalles	peso_kg	decimal(8,2)	validación 0.1–200 en la API	Peso registrado y después pesado
recompensas	tipo	varchar(20)	validación tarjeta_regalo o vale_despensa	Tipo de recompensa; activa=false es baja lógica
canjes	lotes	json		Lotes y cantidades consumidos (FIFO) para devolverlos al cancelar
puntos_lotes	restantes, vence_en	int, timestamp	índice (user_id, vence_en)	Saldo = suma de restantes no vencidos;

Tabla	Columna	Tipo	Restricción / índice	Significado
				vigencia 12 meses
outbox	publicado_en	timestamp nullable	índice	NULL = pendiente de publicar en MongoDB; intentos cuenta fallos

Llaves foráneas en todas las relaciones; las operaciones de varios pasos (registrar entrega con detalles, acreditar, canjear, cancelar) corren en una transacción con bloqueo de fila (SELECT … FOR UPDATE) sobre la entrega, el canje, la recompensa o el usuario, para serializar acreditaciones y consumos concurrentes.

Migraciones

Laravel, en backend/database/migrations/, aplicadas por el Job migraciones en cada despliegue con la cuenta ecocanje_migrador (dueña del esquema). La aplicación usa ecocanje_app (solo DML). Todas las migraciones tienen down(); un rollback de imagen **no** revierte el esquema, por lo que cada cambio de esquema debe ser compatible con la versión anterior (aditivo). Las versiones 0.1.1 → 0.1.3 no cambiaron el esquema (D07).

Consultas representativas

```
-- Tarifa vigente de cada material en una sola consulta (antes una por material, N+1)
SELECT DISTINCT ON (material_id) material_id, puntos_por_kg
FROM tarifas WHERE vigente_desde <= now()
ORDER BY material_id, vigente_desde DESC, id DESC;

-- Saldo disponible de un ciudadano
SELECT coalesce(sum(restantes), 0) FROM puntos_lotes WHERE user_id = $1 AND
vence_en > now();

-- Reporte mensual: kilos y puntos acreditados por material
SELECT m.nombre, sum(d.peso_kg) AS kilos, sum(d.puntos) AS puntos
FROM entrega_detalles d JOIN entregas e ON e.id = d.entrega_id JOIN materiales m ON
m.id = d.material_id
WHERE e.estado = 'acreditada' AND e.created_at >= $1 AND e.created_at < $2
GROUP BY m.nombre ORDER BY kilos DESC;
```

```
-- Lote de eventos pendientes para el worker (varios workers no se pisan)
SELECT * FROM outbox WHERE publicado_en IS NULL ORDER BY created_at, id LIMIT 100 FOR UPDATE SKIP LOCKED;
```

MongoDB (libro de movimientos y fichas)

Colección	Documento	Validación ($jsonSchema, validationAction: error)	Índices
movimientos	{_id: <id del outbox>, tipo, user_id, ocurrido_en, payload{…}, request_id}}	requeridos tipo (string), user_id (int/long), ocurrido_en (string), payload (object)	{user_id: 1, ocurrido_en: -1}, {tipo: 1}
fichas_material	{_id: <id del material>, material_id, atributos{clave: valor}, actualizado_en}	requeridos material_id, atributos (object); claves ^[a-z_]{1,40}$ validadas en la API	_id

Uso visible: el **historial de movimientos** del ciudadano (paginado y filtrable por tipo) y la **ficha técnica** de cada material (atributos libres que no justifican columnas). Consultas:

```
db.movimientos.find({user_id: 34}).sort({ocurrido_en: -1}).skip(0).limit(15)
db.movimientos.countDocuments({"payload.request_id": "corr-1794008231-619"})  // correlación (OBS07)
db.fichas_material.updateOne({_id: 7}, {$set: {atributos: {...}}}, {upsert: true})
```

Consistencia entre motores

- **Fuente de verdad: PostgreSQL.** Toda decisión de negocio (saldo, estados, stock) se toma ahí.
- **No hay atomicidad entre motores.** Cada cambio de estado inserta un evento en outbox **en la misma transacción** que el cambio; si la transacción falla, no hay evento.

- **Publicación idempotente.** El worker (outbox:publicar --loop) publica con updateOne(_id = id del outbox, $setOnInsert, upsert): reintentar nunca duplica el documento. Solo después marca publicado_en.
- **Si MongoDB falla:** la API sigue aceptando escrituras, el evento queda pendiente (intentos + 1) y se publica al volver la base. Medido: entrega registrada con MongoDB caído, publicada 7 s después de su regreso, 1 documento (E11).
- **Si PostgreSQL falla:** la API responde 503 (readiness), no hay escrituras que perder; el worker reintenta con espera creciente sin reiniciarse (0.1.3, E11).
- **Reconstrucción de MongoDB:** como todo movimiento nace en el outbox, el historial puede regenerarse desde PostgreSQL volviendo a marcar eventos como pendientes.

Estrategia de conexiones con varias réplicas

Cada proceso PHP-FPM mantiene **una conexión persistente** a PostgreSQL (PDO::ATTR_PERSISTENT, DB_PERSISTENT=true): 6 procesos × hasta 6 réplicas = 36 conexiones, más el worker y el exportador, por debajo de max_connections = 100. Con una conexión por solicitud, la autenticación scram-sha-256 saturaba la CPU de PostgreSQL (98 % de throttling, p95 6.2 s); con conexiones persistentes se abrió 1 sesión por cada 30 solicitudes y el p95 bajó a 0.99 s (E06, E12). Máximo observado bajo carga: 38 conexiones. MongoDB usa el pool del driver por proceso (máximo observado: 39).

D05. Manual de instalación desde cero

Este manual reconstruye **dev** y **prod** en un equipo limpio. El camino automático es scripts/bootstrap.sh; cada fase indica los comandos equivalentes para ejecutarla a mano. Ningún secreto está en el repositorio: se generan dentro de cada clúster durante la instalación.

1. Prerrequisitos del host

Componente	Versión usada	Para qué
Linux x86_64	Fedora 44, kernel 7.2	host
Docker	29.9.3	nodos de k3d y construcción de imágenes
k3d	v5 (k3d version)	clústeres k3s en

Componente	Versión usada	Para qué
		contenedores
k3s	v1.36.5 (infra/K3S_VERSION)	distribución de Kubernetes
kubectl / helm	cliente compatible con 1.36 / Helm 4	administración y charts
skopeo	cualquiera	consultar etiquetas y digests de imágenes
mkcert	con su CA instalada (mkcert -install)	certificados TLS locales
python3 + PyYAML, openssl, bc	de la distribución	scripts

```
# Nombres locales de ambos ambientes (requiere sudo una sola vez)
echo "127.0.0.1 dev.ecocanje.test prod.ecocanje.test" | sudo tee -a /etc/hosts
mkcert -install              # CA local, solo en este equipo
git clone https://github.com/DaBriNan/EcoCanje.git ecocanje && cd ecocanje && git checkout v1.0.0
scripts/validar.sh           # validación estática antes de tocar clústeres
```

2. Instalación automática

scripts/bootstrap.sh

El script se detiene si k3d-dev o k3d-prod ya existen (nunca borra) y si falta cualquier prerrequisito. Al final imprime el tiempo de cada fase y termina con código distinto de 0 si alguna verificación falla.

3. Fases y comandos equivalentes

Fase	Qué hace	Comandos equivalentes
0	Prerrequisitos, /etc/hosts, CA de mkcert, gancho pre-commit	cp docs/manual/gancho-pre-commit.sh .git/hooks/pre-commit (y gancho-excluir-secretos.txt)
1	Imágenes: usa las existentes o las construye desde la revisión actual	docker build -f backend/Dockerfile.prod --build-arg MONGO_EXT=$(cat infra/MONGODB_EXT_VERSION) --build-arg

Fase	Qué hace	Comandos equivalentes
		APP_VERSION=0.1.3 -t ecocanje-backend:0.1.3 backend; docker build --build-arg VITE_VERSION=0.1.4 -t ecocanje-frontend:0.1.4 frontend
2	Clústeres: dev (1 nodo) y prod (1 server + 2 workers), API y entrada solo en 127.0.0.1, cifrado de Secrets en reposo	scripts/crear-clusters.sh dev; scripts/crear-clusters.sh prod
3	Importar imágenes en cada clúster y compararlas con el id aprobado	k3d image import ecocanje-backend:0.1.3 ecocanje-frontend:0.1.4 -c <amb>; scripts/verificar-imagen.sh <amb> 0.1.3; IMG=ecocanje-frontend scripts/verificar-imagen.sh <amb> 0.1.4
4	Aplicación: namespace, Secrets generados, TLS, migraciones y semilla, despliegue	scripts/desplegar.sh <amb> (llama a crear-secretos.sh y crear-tls.sh, recrea el Job migraciones, aplica infra/<amb> con Kustomize)
5	Observabilidad en prod	kubectl apply -f infra/observabilidad/namespaces.yaml; scripts/crear-secretos-monitoreo.sh k3d-prod; helm upgrade --install kps prometheus-community/kube-prometheus-stack --version 92.17.1 -n monitoring -f infra/observabilidad/kps-values.yaml; helm upgrade --install node-exporter prometheus-community/pr

Fase	Qué hace	Comandos equivalentes
		ometheus-node-exporter --version 4.60.1 -n monitoring-nodos -f infra/observabilidad/node-exporter-values.yaml; scripts/observabilidad.sh prod
6	Logs en prod	scripts/logs.sh prod (Loki 19.1.1 y Alloy 1.14.2; valida la configuración de Alloy antes)
7	Verificación	ver sección 4

Accesos de personas (RBAC por función, D06/D07), después de la instalación: scripts/crear-usuario.sh prod <usuario> <lectura|despliegue|operaciones> (kubeconfig en ~/.local/share/ecocanje-pki/usuarios/, permisos 600).

4. Verificación de cada ambiente

Verificación	Comando	Esperado
Nodos y Pods	kubectl --context k3d-<amb> get nodes; source scripts/ev.sh; pods_sanos k3d-<amb>	nodos Ready; todos los Pods Running/Completed
HTTPS, versión e interfaz	curl https://<amb>.ecocanje.test:<puerto>/api/health/live; .../api/health/ready; .../	{"status":"ok","version":"0.1.3"}, 200, 200
Funcional por API	scripts/probar.sh <amb> tests/roles.sh, tests/flujo.sh, tests/catalogo.sh (esperar 61 s entre pruebas: límite de inicio de sesión)	todas correctas
Funcional por interfaz	scripts/probar.sh <amb> tests/interfaz/correr.sh	23 correctas
Red y exposición	tests/netpol-app.sh <amb>, tests/netpol-frontend.sh <amb>, tests/exposicion.sh	todo lo no permitido, bloqueado

Verificación	Comando	Esperado
RBAC	tests/rbac.sh <amb>	19 correctas
Métricas y alertas (prod)	scripts/prom.sh prod 'up == 0'; scripts/prom.sh prod 'sum(prometheus_rule_group_rules{rule_group=~".*ecocanje.*"}) == 8'; scripts/verificar-metricas.sh prod	sin objetivos caídos; 8 reglas; 44 métricas existentes
Logs (prod)	scripts/loki.sh prod 'sum by (app) (count_over_time({namespace="ecocanje"}[5m]))'	líneas de backend, frontend, bases y worker
Datos	scripts/estado-datos.sh k3d-<amb>	usuarios y entregas semilla; outbox pendientes 0

5. Tiempos medidos (E14)

Corrida	Fecha	Cambio	Total
1	2026-11-06 17:59	primera reconstrucción completa	759 s
2	2026-11-06 18:21	publicación solo en 127.0.0.1	671 s
3	2026-11-06 18:59	k3s v1.36.1 → v1.36.5	638 s

Fases más largas: aplicación en prod (79–198 s, espera de bases y migraciones) y observabilidad (178–194 s). Las tres corridas terminaron verificadas sin pasos manuales adicionales. Diferencias registradas en las evidencias E14.

6. Problemas conocidos al instalar u operar en este host

Síntoma	Causa	Solución
Workers NotReady tras reiniciar el host; docker logs k3d-prod-agent-N muestra failed to find	Docker intercambió las IP de los agentes	docker restart k3d-prod-agent-0 k3d-prod-agent-1 (E11, 5 nov)

Síntoma	Causa	Solución
interface with specified node ip		
HPA sin escalar y evento FailedGetScale: Unauthorized tras suspender el equipo	salto de reloj al reanudar	transitorio (minutos); verificar con kubectl describe hpa backend
Prueba larga con resultados mezclados	el equipo se suspendió	ev y carga.sh usan systemd-inhibit; mantener el equipo con corriente
ErrImageNeverPull	imagen no importada en el clúster	fase 3 (k3d image import)
Navegador rechaza el certificado	CA de mkcert no instalada en ese navegador	mkcert -install con el navegador cerrado

D06. Plan y configuración de seguridad

Controles por capa

Capa	Control	Dónde	Evidencia
Aplicación	Contraseñas con bcrypt; autorización en servidor por rol (middleware Rol); validación con reglas de Laravel (tipos, rangos, unicidad, existencia); consultas por ORM/consultas parametrizadas; límite de 5 intentos de inicio de sesión por minuto y 10	backend/	E09, tests/roles.sh, tests/catalogo.sh

Capa	Control	Dónde	Evidencia
	registros; errores en JSON sin trazas (APP_DEBUG=false); la API responde 401 en JSON aunque falte Accept		
Navegador y API	TLS de la CA local (mkcert) en Traefik; mismo origen (sin CORS); token Bearer en sessionStorage sin cookies (CSRF no aplica); token de 8 h; CSP default-src 'self' sin scripts en línea, frame-ancestors 'none', HSTS, nosniff, no-referrer	frontend/seguridad.conf, infra/base/ingreso.yaml	E02, captura de cabeceras
Contenedores	No root (uid fijo), allowPrivilegeEscalation: false, capabilities: drop [ALL], seccompProfile: RuntimeDefault, sistema de archivos de solo lectura (escritura solo en emptyDir acotados), límites de CPU y memoria	todos los manifiestos y valores de Helm	admisión Pod Security restricted (rechazaría lo contrario)
Kubernetes	Pod Security Admission restricted en ecocanje y	infra/base/namespace.yaml, cuentas.yaml, rbac.yaml,	E09 tests/rbac.sh 19/19

Capa	Control	Dónde	Evidencia
	monitoring; excepción privileged solo en monitoring-nodos (node-exporter necesita el host); cuentas de servicio por carga sin token montado; RBAC por función para personas; cuota y LimitRange	cuotas.yaml	
Secretos	Generados dentro de cada clúster por script (nunca en Git ni en imágenes), cifrado de Secrets en reposo (--secrets-encryption), montados como archivos 0440 o variables de entorno; llaves TLS, de respaldo y de usuarios fuera del repositorio (~/.local/share/ecocanje-pki, permisos 700/600); gancho pre-commit y validar.sh buscan secretos	scripts/crear-secretos*.sh, crear-tls.sh, crear-usuario.sh	SEC01, SEC02, CI
Datos	Cuentas de base de datos por función, sin rol	infra/base/postgres.yaml, mongo.yaml, scripts/exportar-	E09 (usuarios de BD), D09

Capa	Control	Dónde	Evidencia
	administrativo para la aplicación (ver tabla); respaldos cifrados AES-256 fuera del clúster	respaldos.sh	
Red	Denegación inicial de entrada y salida en ecocanje; flujos permitidos explícitos (ver matriz); bases sin Ingress ni NodePort; API de Kubernetes y entrada publicadas solo en 127.0.0.1	infra/base/redes.yaml, frontend.yaml, scripts/crear-clusters.sh	E08 (backend, frontend, exposición 6/6)
Suministro	Imágenes por digest (o tag + digest); charts con versión fija; imágenes propias aprobadas por id y promovidas sin reconstruir; análisis con Trivy y tratamiento de hallazgos; CI en cada cambio	kustomization.yaml, infra/imagen-aprobada-*.id, scripts/vulnerabilidades.sh, .github/workflows/ci.yml	SUP01–SUP04
Infraestructura	k3s actualizado a v1.36.5 por CVE de Traefik; kubeconfig de administración con permisos 600; firewall de host	infra/K3S_VERSION, infra/firewall/	E14, SUP03

Capa	Control documentado	Dónde	Evidencia

Matriz RBAC (personas)

Acción en ecocanje	lectura	despliegue	operaciones	administrador
Ver Pods, logs, eventos, Deployments, HPA, Jobs, Ingress, NetworkPolicy	sí	sí	sí	sí
Leer Secrets	no	no	no	sí
Cambiar imagen, escalar, reiniciar o deshacer rollouts; borrar Pods	no	sí	sí	sí
Crear Jobs (migraciones, respaldos manuales)	no	sí	sí	sí
exec y port-forward (diagnóstico, respaldos)	no	no	sí	sí
Modificar NetworkPolicy, RBAC, cuotas, PVC	no	no	no	sí
Otros namespaces (monitoring, kube-system)	no	no	no	sí

Los permisos se otorgan a grupos (ecocanje:lectura, ecocanje:despliegue, ecocanje:operaciones); la identidad viaja en un certificado de cliente emitido por la API CertificateSigningRequest, con vigencia de 30 días. Kubernetes no revoca certificados de cliente: para retirar un acceso se quita el grupo del RoleBinding o se espera el vencimiento. operaciones puede leer el entorno de un contenedor con exec; por eso esa función se asigna solo a quien opera respaldos e incidentes. Las cargas usan cuentas de servicio propias (backend, postgres, mongo, respaldo, exportador) sin token montado: ninguna necesita la API de Kubernetes. Las excepciones son Alloy y Prometheus, que leen la API con roles de solo lectura de sus charts.

Cuentas de base de datos (mínimo privilegio)

Cuenta	PostgreSQL	MongoDB	Usa
ecocanje_admin / root	superusuario (solo inicialización y diagnóstico)	root	no la usa ninguna carga
ecocanje_migrador	dueño del esquema public	readWrite + dbAdmin en ecocanje	Job de migraciones
ecocanje_app	SELECT, INSERT, UPDATE, DELETE en tablas, uso de secuencias; sin DDL	readWrite en ecocanje	backend y worker
ecocanje_backup	pg_read_all_data	backup	CronJobs de respaldo
ecocanje_monitor	pg_monitor	clusterMonitor	exportadores

PUBLIC sin privilegios sobre la base ni el esquema; autenticación scram-sha-256 (se retiró trust, hallazgo de E09 del 27 oct).

Flujos de red permitidos

Origen	Destino	Puerto	Política
Todos los Pods de ecocanje	CoreDNS (kube-system)	53 UDP/TCP	permitir-dns
Traefik (kube-system)	backend	8080	backend-ingreso

Origen	Destino	Puerto	Política
Traefik (kube-system)	frontend	8080	frontend-ingreso
Pods con postgres-egress (backend, worker, migraciones, respaldo, exportador)	postgres	5432	clientes-postgres-egreso + postgres-ingreso
Pods con mongo-egress	mongo	27017	clientes-mongo-egreso + mongo-ingreso
Prometheus (monitoring)	exportadores	9187, 9216	exportadores-desde-prometheus
Pods de monitoring	Pods de monitoring	todos	solo-ingreso-del-mismo-namespace
Cualquier otro	—	—	denegar-todo (entrada y salida)

Demostración A/B en E08: el mismo cliente es rechazado por la política de entrada, conecta con una regla temporal que lo permite y vuelve a ser rechazado al retirarla; DNS y HTTPS sirven de control.

Firewall

Docker inserta sus reglas antes que el firewall del host, de modo que un puerto publicado en 0.0.0.0 queda alcanzable desde la LAN aunque firewalld esté activo. El control efectivo es publicar en 127.0.0.1 (crear-clusters.sh); antes de corregirlo la API de Kubernetes y la entrada respondían desde la Wi-Fi y desde ZeroTier (E08, FALLA 2/6 → 6/6). La zona de firewalld infra/firewall/ecocanje.xml (DROP; permite 9993/udp de ZeroTier y, como excepción documentada, 10250 y 6443 del otro clúster que comparte el host) queda **documentada y no activada** por decisión del propietario del equipo. Orden de evaluación: puertos permitidos de la zona y después el target DROP; las respuestas a conexiones salientes siguen permitidas. Se activa, revierte y exporta con infra/firewall/aplicar.sh.

Secretos (creación, rotación y recuperación)

- **Creación:** scripts/crear-secretos.sh <contexto> genera contraseñas aleatorias con openssl rand y crea los Secrets solo si no existen; nunca imprime valores.
- **Rotación de una contraseña de base de datos:** generar el valor nuevo, aplicarlo en el motor con la cuenta administrativa (ALTER ROLE … PASSWORD o db.changeUserPassword) leyendo el valor desde un archivo, actualizar el Secret con kubectl create secret … --dry-run=client -o yaml | kubectl apply -f - y reiniciar las cargas (rollout restart). Rotación completa ejecutada en dev tras una exposición accidental (SEC02, 27 oct).
- **Rotación de todo un ambiente:** recrearlo con bootstrap.sh regenera todos los Secrets, el token de unión de k3s y la contraseña de Grafana (así se rotó el K3S_TOKEN expuesto en una sesión el 5 nov).
- **Recuperación:** los Secrets no se respaldan en claro; los respaldos de datos se cifran con ~/.local/share/ecocanje-pki/respaldo.key, que debe guardarse fuera del equipo (D09).

Vulnerabilidades (Trivy 0.76.1, 6 nov) y tratamiento

Hallazgo CRITICAL	Componente	Decisión
CVE-2026-111497 (brotli)	frontend <= 0.1.0	**Corregido**: apk upgrade sobre la base; frontend 0.1.1+ con 0 CRITICAL y 0 HIGH
CVE-2026-111497 (brotli)	sidecar nginx del backend (imagen oficial 1.30.6)	**Aceptado**: no hay compilación corregida publicada; nginx no enlaza brotli (solo image_filter, no cargado, y curl/freetype, no usados)
CVE-2025-68121 (Go en gosu)	postgres, mongo oficiales	**Aceptado**: corren como uid 70/999, gosu no se ejecuta y no usa TLS
CVE-2026-65600, CVE-2026-88007	Traefik 3.6.13	**Corregido**: k3s v1.36.5 trae Traefik 3.7.13 (0 CRITICAL)
CVE-2026-33186,	metrics-server, klipper-	**Corregido** con k3s

Hallazgo CRITICAL	Componente	Decisión
CVE-2025-68121	helm de k3s v1.36.1	v1.36.5 (0 CRITICAL)

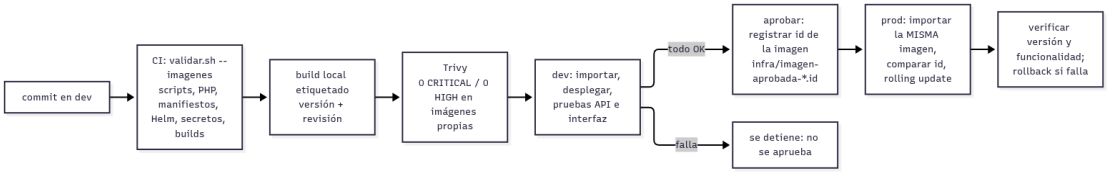
Las imágenes propias (backend 0.1.3, frontend 0.1.4) tienen 0 CRITICAL y 0 HIGH. Los HIGH de componentes de terceros quedan como riesgo residual y se revisan con cada actualización.

Riesgos residuales

Riesgo	Mitigación actual	Siguiente paso
Host único compartido por ambos ambientes y el otro clúster	publicación solo local; Pod Security; cuotas	hosts separados
Imagen base del backend `php:8.4-fpm-alpine` sin digest	imagen aprobada por id y promovida sin reconstruir; Trivy 0/0	fijar `php:8.4.x-fpm-alpine@sha256` en la próxima versión
HIGH en imágenes de terceros	versiones fijadas, análisis periódico	actualizar charts y k3s en cada parche
Certificados de cliente no revocables	vigencia de 30 días	OIDC con revocación
`exec` de la función operaciones expone variables de entorno	función restringida a quien opera	credenciales solo como archivos, no como variables
Firewall de host no activado	publicación solo en loopback	activar `infra/firewall/aplicar.sh` si el equipo se expone
Sin auditoría de la API de Kubernetes	logs de eventos en Loki	activar `--audit-log-path` en k3s

D07. Publicación, promoción, actualización, rollback y accesos

Ciclo de una versión



diagram

- **Artefacto inmutable.** La imagen que llega a prod es la misma que se probó en dev: verificar-imagen.sh compara el id de la imagen en cada nodo con el registrado al aprobar; si difiere, la promoción se detiene.
- **Solo cambia la configuración del ambiente.** Las diferencias entre dev y prod viven en infra/dev e infra/prod (réplicas, HPA, PDB, reparto, host); la imagen y la base son las mismas.
- **Sin registro de imágenes.** En el laboratorio las imágenes se importan con k3d image import (imagePullPolicy: IfNotPresent); en un entorno real se publicarían por digest en un registro privado.

Procedimientos

Tarea	Comando	Compuertas
Promover el backend	scripts/actualizacion.sh prod <nueva> <anterior>	imagen aprobada; versión y prueba de roles en cada fase; sonda HTTPS sin errores
Promover el frontend	scripts/promover-frontend.sh <versión>	Trivy 0/0; prueba de interfaz en dev antes de aprobar; misma imagen en prod; prueba de interfaz en prod
Desplegar un ambiente	scripts/desplegar.sh <dev|prod>	recrea el Job migraciones; espera su éxito y el rollout
Rollback de imagen	kubectl -n ecocanje rollout undo deploy/backend deploy/outbox-worker (o	prueba de versión y funcionalidad

Tarea	Comando	Compuertas
	deploy/frontend) y después restaurar el estado declarado en Git	

La actualización es gradual (maxSurge: 1, maxUnavailable: 0), con readiness que valida PostgreSQL y preStop de 5 s para que el Pod saliente siga atendiendo mientras se retira de los endpoints. Sin ese preStop se perdían 3 solicitudes por fase (E12, primer intento); con él, 0.

Migraciones y rollback

Un rollback de imagen **no** revierte el esquema. Política: cada migración debe ser compatible con la versión anterior de la aplicación (solo aditiva: tablas o columnas nuevas, nulas o con valor por defecto). Un cambio destructivo se hace en dos versiones: primero se deja de usar la columna, después se elimina. Si una migración no es compatible, su plan de reversión es php artisan migrate:rollback --step=1 (todas tienen down()) ejecutado como Job con la cuenta ecocanje_migrador **antes** del rollout undo, previo respaldo (scripts/respaldar.sh prod). Entre 0.1.1 y 0.1.3 no hubo cambios de esquema.

Historial de versiones

Imagen	Versión	Cambio	Aprobada
backend	0.1.1	imagen construida el 2 nov desde la revisión 978939b y desplegada en dev y prod (la 0.1.0, del 27 oct, introdujo la imagen de producción y los endpoints de salud)	id ac2590b7b87d3
backend	0.1.2	conexiones persistentes a PostgreSQL; versión visible;	94410517655dd

Imagen	Versión	Cambio	Aprobada
		caché de rutas y eventos	
backend	0.1.3	correlación X-Request-Id; worker resiliente; sin N+1; 401 en JSON; desempate de tarifas	41f344a148ebb
frontend	0.1.0	SPA inicial	6451729c69f9b
frontend	0.1.1	apk upgrade (brotli, CVE crítico)	ba0a23b71c651
frontend	0.1.2	validación en el navegador; búsqueda por ciudadano	453c385884cdc
frontend	0.1.3	descarta respuestas de consultas no vigentes	79b8fa604e6dd
frontend	0.1.4	un 401 tardío ya no cierra una sesión nueva	06cef4d5cff81

Pruebas de actualización y rollback: 0.1.1 → 0.1.2 → 0.1.1 → 0.1.2 y 0.1.2 → 0.1.3 → 0.1.2 → 0.1.3, ambas con 0 errores en la sonda (E12). En la segunda, la prueba de roles falla justo en las fases con 0.1.2 por el defecto del 401 corregido en 0.1.3: confirma que el rollback restauró el comportamiento anterior.

Lección de procedimiento. El frontend 0.1.3 se promovió a prod aunque su prueba de interfaz en dev había fallado: la cadena de comandos filtraba la salida con grep y ocultó el código de salida (anotado en la evidencia DEP05). Desde entonces la promoción usa promover-frontend.sh, donde cada paso es una compuerta (set -euo pipefail).

Administración de accesos

Tarea	Procedimiento
Alta	scripts/crear-usuario.sh <amb> <usuario> <lectura|despliegue|operaciones>; entregar el kubeconfig por un canal seguro
Cambio de función	emitir un certificado nuevo con la otra función; el anterior vence en 30 días
Baja inmediata	quitar el grupo del RoleBinding (afecta a todos los de esa función) o recrear la CA del clúster; en un entorno real, OIDC
Revisión	kubectl get csr; kubectl auth can-i --list --as=<usuario> --as-group=ecocanje:<función> -n ecocanje
Administrador	kubeconfig de k3d en ~/.kube/config (permisos 600); solo el propietario del equipo
Aplicación	usuarios en la base (rol ciudadano u operador); el alta de operadores se hace por semilla o por el administrador de la base

D08. Operación, observabilidad, alertas e incidentes

Componentes (solo prod; dev usa diagnóstico básico con kubectl y metrics-server)

Señal	Componente	Acceso
Métricas	Prometheus (kube-prometheus-stack 92.17.1, retención 2 días / 2 GB), kube-state-metrics, node-exporter, métricas de Traefik,	scripts/prom.sh prod '<PromQL>'

Señal	Componente	Acceso
	postgres-exporter v0.20.1 y mongodb_exporter 0.53.0 con usuarios monitor	
Paneles	Grafana con el panel “EcoCanje - produccion” (33 paneles, ConfigMap generado desde infra/observabilidad/prod/paneles/generar.py) y la fuente Loki	kubectl -n monitoring port-forward svc/kps-grafana 3000:80; usuario en el Secret grafana-admin
Alertas	8 reglas PrometheusRule + Alertmanager **sin receptores externos** (no se envían datos a terceros)	kubectl -n monitoring port-forward svc/kps-alertmanager 9093
Logs	Loki 3.7.9 (retención 48 h) y Alloy v1.21.1: logs de ecocanje, monitoring y kube-system más eventos de Kubernetes	scripts/loki.sh prod '<LogQL>'; Grafana → Explore
Correlación	X-Request-Id generado o propagado por nginx, devuelto al cliente, en el log JSON de Laravel, en el evento del outbox, en el log del worker y en el documento de MongoDB	scripts/loki.sh prod '{namespace="ecocanje"} |= "<id>"'

Cobertura frente al PDF (§10)

Objeto	Indicadores en el panel o en consultas
Nodos	Ready, CPU, memoria disponible, disco de /var/lib/rancher/k3s, red rx/tx, CPU asignable libre
Kubernetes	réplicas deseadas frente a disponibles, HPA actual/deseado/máximo, Pods no

Objeto	Indicadores en el panel o en consultas
	listos, Pending y reinicios, réplicas por nodo, eventos en Loki
Aplicación	tráfico por código, tasa de 5xx, p95/p50 (Traefik), salud de dependencias (/api/health/ready, /api/health/deps), logs con fecha y request_id
PostgreSQL	pg_up, conexiones por estado frente a max_connections, transacciones/s, tamaño de la base
MongoDB	mongodb_up, conexiones actuales/disponibles, operaciones/s, latencia media, tamaño de datos
Red y seguridad	días para expirar el certificado TLS, códigos 4xx/5xx en Traefik, eventos y rechazos en logs
Respaldos	horas desde el último respaldo exitoso, duración del último Job, Jobs fallidos (más el log respaldo_ok con tamaño)

Alertas

Alerta	Consulta	Umbral / duración	Severidad	Acción
AplicacionIndisponible	kube_deployment_status_replicas_available{deployment="backend"}	< 1 durante 1 min	critical	escenario 1
NodoNoDisponible	kube_node_status_condition{condition="Ready",status="true"}	== 0 durante 1 min	critical	escenario 3
BaseDeDatosCaida	up{job="ecocanje/exportadores"}, pg_up, mongodb_up	== 0 durante 1 min	critical	escenario 4

Alerta	Consulta	Umbral / duración	Severidad	Acción
TasaErroresElevada	5xx / total en el servicio del backend (Traefik, 5 min)	> 5 % con > 0.05 req/s, 2 min	warning	escenario 2
SaturacionRecursos	CPU del contenedor php / su límite, por Pod (5 min)	> 85 % durante 5 min	warning	escenario 5
NodoMemoriaBaja	MemAvailable / MemTotal	< 10 % durante 5 min	warning	escenario 6
RespaldoFallido	kube_job_status_failed{job_name=~"(respaldo|manual)-.*"}	> 0 durante 1 min	critical	escenario 7
RespaldoVencido	tiempo desde el último respaldo exitoso del CronJob (o desde su creación si nunca tuvo éxito)	> 25 h durante 5 min	warning	escenario 7

Probadas de forma controlada (E10): AplicacionIndisponible (backend a 0 réplicas: pending a los 40 s, firing a los 101 s, resuelta 21 s después de restaurar) y RespaldoFallido (Job sin credencial: firing a los 72 s, resuelta 20 s después de repetir el respaldo); ambas llegaron a Alertmanager y se resolvieron. RespaldoFallido también se disparó por un incidente real (respaldos del 3 nov con DeadlineExceeded mientras el equipo estaba suspendido, OBS05). **Umbral efectivo:** con scrape de 30 s y for: 1m, una caída avisa a partir de ~96 s; en E11 una caída de PostgreSQL de 72 s dejó las alertas en pending y se resolvieron sin notificar (intencional, evita ruido).

Operación diaria

Tarea	Comando
Estado general	kubectl --context k3d-prod get nodes; source scripts/ev.sh; pods_sanos k3d-prod
Objetivos caídos y alertas activas	scripts/prom.sh prod 'up == 0'; scripts/prom.sh prod 'ALERTS{alertstate="firing"}'
Datos y outbox	scripts/estado-datos.sh k3d-prod
Errores recientes del backend	scripts/loki.sh prod '{app="backend", container="php"} | json | level_name="ERROR"' 1h 20
Seguir una solicitud	scripts/loki.sh prod '{namespace="ecocanje"} |= "<X-Request-Id>"' 1h 50
Métricas que usan alertas y panel existen	scripts/verificar-metricas.sh prod

Manual de incidentes

Cada escenario: **diagnóstico → acciones → verificación → escalamiento**. Registrar línea de tiempo con ev.

1. Aplicación indisponible (AplicacionIndisponible; la interfaz o /api responden 503) - Diagnóstico: kubectl -n ecocanje get deploy,pods -l app=backend; kubectl -n ecocanje describe pod <pod> (eventos); curl …/api/health/ready (503 postgres_no_disponible → escenario 4); logs {app="backend"} en Loki. - Acciones: si las réplicas son 0, kubectl -n ecocanje scale deploy/backend --replicas=2; si CrashLoop, revisar el log de php y hacer rollback (D07); si Pending, revisar cuota y nodos (escenarios 3 y 5). - Verificación: /api/health/ready = 200; alerta resuelta; tests/roles.sh. - Escalamiento: si persiste tras el rollback, administrador (revisión de nodos y almacenamiento).

2. Tasa de errores elevada (TasaErroresElevada) - Diagnóstico: códigos en el panel; scripts/loki.sh prod '{app="backend", container="php"} | json | level_name="ERROR"'; tomar un request_id y seguirlo; revisar dependencias con /api/health/deps (rol operador). - Acciones: si coincide con una versión nueva, rollback (D07); si es una dependencia, escenario 4. - Verificación: tasa de 5xx < 1 % en 10 min. - Escalamiento: desarrollador con los request_id afectados.

3. Nodo no disponible (NodoNoDisponible) - Diagnóstico: kubectl get nodes; docker ps --filter name=k3d-prod; docker logs k3d-prod-agent-N | grep -i shutdown. - Acciones: si el log muestra failed to find interface with specified node ip (tras reiniciar el host, Docker intercambió las IP), docker restart k3d-prod-agent-0 k3d-prod-agent-1; si el contenedor no existe, recrear el ambiente (D05). Considerar que Prometheus puede estar en el nodo caído: si no hay métricas, diagnosticar con kubectl. - Verificación: nodos Ready; pods_sanos; up == 0 sin resultados. - Escalamiento: administrador del host. Caso real: 5 nov (E11), recuperado con el reinicio de los agentes.

4. Base de datos caída (BaseDeDatosCaida) - Diagnóstico: kubectl -n ecocanje get sts,pods -l 'app in (postgres,mongo)'; eventos; ¿el nodo de su volumen está Ready o acordonado? (local-path ata la base a su nodo). - Acciones: PostgreSQL: la API queda en 503 sin reiniciarse (liveness no depende de la base); recuperar el Pod (kubectl -n ecocanje rollout status sts/postgres; si el nodo está acordonado, uncordon). MongoDB: la API sigue aceptando escrituras y el outbox acumula; al volver la base se publica solo. - Verificación: pg_up / mongodb_up = 1; scripts/estado-datos.sh con outbox pendientes 0; documento en MongoDB por request_id. - Escalamiento: si el volumen se perdió, restauración desde respaldo (D09). Medido (E11): MongoDB 28 s caído → publicado 7 s después, sin duplicados; PostgreSQL 60 s caído → servicio de vuelta a los 72–75 s, 0 reinicios de backend y worker.

5. Saturación de recursos (SaturacionRecursos) - Diagnóstico: kubectl -n ecocanje get hpa backend; kubectl top pods; panel de CPU y throttling; ¿el HPA está en 6? - Acciones: si el HPA no alcanza el máximo, revisar kubectl describe hpa (métricas ausentes); si está en 6, el límite es la CPU de los Pods o del host (D03): limitar la carga o ampliar capacidad. No desplegar migraciones con el HPA en 6 (excederían la cuota). - Verificación: CPU por debajo del 85 % del límite; p95 < 1 s. - Escalamiento: planeación de capacidad.

6. Memoria baja en un nodo (NodoMemoriaBaja) - Diagnóstico: kubectl top pods -A --sort-by=memory. En k3d todos los nodos ven la memoria del host completo. - Acciones: liberar memoria en el host (otras aplicaciones, docker system df) o reducir límites. - Verificación: memoria disponible > 10 %.

7. Respaldo fallido o vencido (RespaldoFallido, RespaldoVencido): ver D09.

8. Equipo suspendido o reiniciado - Síntomas: pruebas con huecos de horas; HPA con FailedGetScale: Unauthorized por unos minutos; Grafana sin pasar su sonda mientras se pone al día; respaldos con DeadlineExceeded. - Acciones:

verificar nodos (escenario 3), pods_sanos, up == 0; repetir respaldos si fallaron. - Prevención: las evidencias y la carga corren bajo systemd-inhibit.

9. Mantenimiento de un worker (drain/uncordon) - Procedimiento: scripts/fallos.sh prod drain (NODO=...): drain respetando PDB, uncordon y rollout restart de backend y frontend para reequilibrar. Medido (E11): 0 solicitudes fallidas; backend y frontend mantienen 2 réplicas en el otro worker; la base cuyo volumen está en ese nodo queda detenida hasta el uncordon.

D09. Respaldos, restauración y continuidad

Objetivos (definidos el 2 nov, antes de medir; docs/rpo-rto.md)

- **RPO 24 h**: respaldos diarios; la pérdida máxima aceptable es un día de operaciones.
- **RTO 30 min**: desde el inicio de la restauración hasta que la aplicación atiende con los datos restaurados.

Respaldos

Motor	Método (consistente)	Programación	Verificación al respaldar	Retención
PostgreSQL	pg_dump -Fc con la cuenta ecocanje_backup (pg_read_all_data): instantánea transaccional	CronJob respaldo-postgres 02:00 America/Mexico_City	pg_restore --list del archivo, SHA-256 escrito y comprobado; línea respaldo_ok archivo=... bytes=... fecha=...	7 archivos en el PVC respaldos
MongoDB	mongodump --archive --gzip con la cuenta ecocanje_backup (rol backup)	CronJob respaldo-mongo 02:10	tamaño mínimo (> 100 bytes) y SHA-256; respaldo_ok. La validez completa se comprueba al	7 archivos

Motor	Método (consistente)	Programación	Verificación al respaldar	Retención
			restaurar	

El respaldo de PostgreSQL espera con pg_isready hasta poder conectarse (cambio tras el fallo del 2 nov); el de MongoDB no tiene esa espera y depende del reintento del Job (backoffLimit: 1), una asimetría por corregir. Ambos corren como no root, con concurrencyPolicy: Forbid, activeDeadlineSeconds: 900 y backoffLimit: 1. No se copian archivos de datos en caliente.

Fuera del clúster: scripts/exportar-respaldos.sh <amb> copia los respaldos a ~/ecocanje-respaldos/<amb>/, cifrados con AES-256 (llave en ~/.local/share/ecocanje-pki/respaldo.key, permisos 600) y con su SHA-256. La llave debe guardarse además fuera del equipo: sin ella los respaldos exportados no se pueden leer.

Restauración (scripts/restaurar.sh <amb>)

Restaura **en el namespace aislado recuperacion** (infra/recuperacion), nunca sobre el origen: crea bases nuevas, carga el último respaldo de cada motor, levanta la aplicación contra ellas y compara con el origen: conteos por tabla y colección, relaciones sin huérfanos, índices, validador de MongoDB, salud de la aplicación y autorización por rol con los usuarios restaurados.

Resultados medidos (prod, E13)

Medición	Resultado	Objetivo
Restauración de ambas bases en espacio aislado	6/6 verificaciones (conteos, relaciones, índices, validador, aplicación, roles)	—
RTO	**60 s** (2 nov 15:59) y **27 s** (2 nov 18:32)	1800 s: cumple
RPO (prueba con marcador)	Un dato escrito después del respaldo existe en el origen y **no** en la restauración: la pérdida real es todo lo escrito desde el último respaldo; con respaldos diarios, hasta 24 h	24 h: cumple por diseño

Medición	Resultado	Objetivo
Antigüedad del respaldo usado	2364 s al restaurar	< 86400 s

El primer intento de la prueba de RPO falló (3/4) porque la verificación consultó la aplicación restaurada antes de que estuviera lista; se repitió con espera activa (4/4). Ambas corridas se conservan.

Fallos de respaldo registrados

Fecha	Fallo	Causa	Tratamiento
2 nov 14:35	respaldo de PostgreSQL y exportación con FALLA (MongoDB respaldó bien en el mismo momento)	pg_dump recibió *Connection refused* al iniciar el Job con la base sana; causa probable: la regla de red del Pod nuevo aún no estaba programada por kube-router (mismo efecto medido en E10). La exportación copió un .tmp incompleto	espera con pg_isready antes de pg_dump (absorbe el retraso); la exportación excluye .tmp; repetición OK
3 nov	respaldos programados con DeadlineExceeded	el equipo estaba suspendido	alerta RespaldoFallido real (OBS05); respaldo repetido; Jobs fallidos eliminados tras el diagnóstico

Continuidad

Escenario	Respuesta	Pérdida esperada
Caída de un Pod de aplicación	el Deployment lo recrea (6.5 s medido)	ninguna

Escenario	Respuesta	Pérdida esperada
Caída de la base no relacional	outbox acumula; publica al volver	ninguna (retraso del historial)
Caída de PostgreSQL	API en 503; recuperación automática del Pod	ninguna si el volumen sobrevive
Pérdida del volumen de una base	restaurar.sh desde el último respaldo; para MongoDB, además, republicar desde el outbox	hasta 24 h (PostgreSQL)
Pérdida del clúster	bootstrap.sh (11 min medidos) + restauración del respaldo exportado	hasta 24 h
Pérdida del host	nuevo host con los prerrequisitos (D05), repositorio, respaldos exportados y su llave	hasta 24 h; requiere copia externa de respaldos y llave

Limitaciones: respaldos y clúster comparten el host y su disco; una sola réplica por motor; los respaldos exportados deben copiarse a otro medio para cubrir la pérdida del host. Una reducción del RPO requeriría respaldos más frecuentes o replicación continua (archivado de WAL en PostgreSQL).

D10. Plan de pruebas, resultados, evidencias y conclusiones

Método

Cada prueba se ejecuta con ev (scripts/ev.sh): guarda en docs/evidencias/ el identificador, fecha, ambiente, host, commit, comando, resultado esperado, salida (con valores sensibles ocultos) y resultado, y añade una fila a INDEX.md. Las pruebas terminan con código distinto de 0 si el estado es malo. **Las corridas fallidas se conservan** con su diagnóstico y la repetición se registra aparte; las mediciones afectadas por causas externas se marcan como invalidadas, sin borrarlas.

Al 6 nov: **230 evidencias** (192 OK, 35 fallidas con diagnóstico, 3 invalidadas); de ellas, 129 de la matriz E01–E14.

Tipo	Herramienta	Ubicación
Funcionales por API (válidos, inválidos, duplicados, sin permiso, transiciones)	bash + curl	tests/roles.sh, flujo.sh, catalogo.sh
Interfaz con navegador real (escritorio y celular)	Playwright 1.64.1 en Docker	tests/interfaz/ (23 verificaciones)
Red y exposición	kubectl, nc, curl	tests/netpol*.sh, exposicion.sh
RBAC	certificados de cliente	tests/rbac.sh
Carga y escalado	k6 2.2.0 en Docker, series de Prometheus	tests/carga/, scripts/carga.sh
Fallos	kubectl, sonda HTTPS a 5 Hz	scripts/fallos.sh, tests/recreacion-datos.sh
Actualización y rollback	sonda, versión, prueba de roles	scripts/actualizacion.sh, promover-frontend.sh
Observabilidad	PromQL, LogQL, Alertmanager	scripts/prom.sh, loki.sh, provocar-alerta.sh, tests/correlacion.sh
Respaldos y recuperación	pg_dump, mongodump, namespace aislado	scripts/respaldar.sh, restaurar.sh, tests/rpo.sh
Estáticas y suministro	shellcheck, php -l, kubeconform, helm, Trivy, CI	scripts/validar.sh, vulnerabilidades.sh, .github/workflows/ci.yml

Matriz de aceptación

ID	Condición (PDF §12)	Resultado	Estado
E01	Dos contextos y topologías distintas; nodos y componentes sanos	dev 1 nodo, prod 1+2; nodos Ready y Pods sanos tras cada reconstrucción; incidente real de workers tras reiniciar el host	Cumple

ID	Condición (PDF §12)	Resultado	Estado
		(diagnosticado y resuelto)	
E02	URLs HTTPS distintas y datos independientes	dev.ecocanje.test:8444 y prod.ecocanje.test:8445 con TLS; tests/independencia.sh sin datos, credenciales ni volúmenes compartidos	Cumple
E03	CRUD de tres entidades, roles y validaciones desde interfaz y API	API: flujo 22/22, catálogo 16/16 (incluye regresión del empate de tarifas); interfaz 23/23 en dev y prod (validación en cliente y servidor, búsqueda, sesión expirada), estable en 3 corridas	Cumple
E04	Uso funcional y persistente de ambas bases con consultas trazables	historial y fichas en MongoDB; outbox sin duplicados; correlación de una solicitud hasta el documento (OBS07)	Cumple
E05	Recreación de Pods de datos sin pérdida	huella de 6 métricas en PostgreSQL y 2 en MongoDB idéntica	Cumple

ID	Condición (PDF §12)	Resultado	Estado
		tras recrear ambos Pods (8 s c/u, mismo PVC) en dev y prod; la primera corrida dio un falso OK y se invalidó	
E06	HPA crece y disminuye; series y carga adjuntas	2 → 6 → 2 (333 s); p95 0.99 s, 0 % errores, 124 rps; sin HPA p95 12.6 s, 13 % errores; CSV en evidencias/carga/. Una corrida invalidada por suspensión del equipo; la 0.1.1 no cumplía (cuello en PostgreSQL)	Cumple (p95 con poco margen)
E07	Varias réplicas atienden; distribuidas entre workers	60 solicitudes: 30/30 entre dos réplicas en workers distintos (logs de nginx)	Cumple
E08	Conexiones permitidas y denegadas; DNS y servicios como control	A/B con regla temporal para backend y frontend; el frontend no alcanza bases ni backend; exposición solo en 127.0.0.1 (6/6 tras corregir 0.0.0.0)	Cumple
E09	Acceso denegado	roles de la API	Cumple

ID	Condición (PDF §12)	Resultado	Estado
	por rol de aplicación, RBAC y usuario de base	(incluye 401 sin Accept); RBAC 19/19 con certificados reales; usuarios de BD de mínimo privilegio	
E10	Paneles con nodos, servicios y ambos motores; dos alertas activadas y resueltas	panel de 33 paneles con datos; AplicacionIndisponible y RespaldoFallido activadas y resueltas en Prometheus y Alertmanager; RespaldoFallido real (OBS05)	Cumple
E11	Fallo de Pod y mantenimiento de worker con tiempos e impacto	Pod: 6.5 s, 0 errores; drain/uncordon: 0 errores, 2 réplicas mantenidas tras corregir el reparto; MongoDB caído: sin pérdida; PostgreSQL caído: 72–75 s, 0 reinicios	Cumple
E12	Actualización y rollback con versión y funcionalidad	backend 0.1.1→0.1.2→0.1.1→0.1.2 y 0.1.2→0.1.3→0.1.2→0.1.3 con 0 errores; frontend hasta 0.1.4 con	Cumple

ID	Condición (PDF §12)	Resultado	Estado
		compuertas	
E13	Respaldo y restauración de ambas bases; RPO y RTO	restauración aislada 6/6; RTO 60 s y 27 s; pérdida demostrada de un dato posterior al respaldo (RPO <= 24 h)	Cumple
E14	Reconstrucción desde recursos limpios con el manual	759 s, 671 s y 638 s (k3s v1.36.5); verificadas, sin pasos manuales adicionales	Cumple

Hallazgos que las pruebas encontraron y se corrigieron

Hallazgo	Detectado por	Corrección
PostgreSQL saturado por una conexión scram por solicitud (p95 6.2 s)	E06	conexiones persistentes (backend 0.1.2)
3 solicitudes perdidas por rollout	sonda de E12	preStop de 5 s
Worker del outbox en CrashLoopBackOff si cae PostgreSQL	eventos en Loki tras E11	reintento con espera creciente (0.1.3)
API respondía 500 sin token ni Accept	correlación por request_id	redirectGuestsTo y JSON en api/* (0.1.3)
Tarifas del mismo segundo: ganaba la anterior (también en el cálculo de puntos)	prueba de interfaz	desempate por id (0.1.3)
Consulta N+1 en el catálogo	revisión del controlador	DISTINCT ON (0.1.3)
Réplicas en Pending	E11	nodeTaintsPolicy: Honor,

Hallazgo	Detectado por	Corrección
durante el drain y 4 réplicas en un solo worker tras un rollout		matchLabelKeys
API de Kubernetes y entrada alcanzables desde la LAN y la VPN	revisión de puertos publicados	publicación en 127.0.0.1
CVE crítico en la base del frontend y en Traefik	Trivy	apk upgrade (frontend 0.1.1); k3s v1.36.5
Validación solo en servidor; sin búsqueda; respuestas viejas pisaban a las nuevas; un 401 tardío cerraba una sesión nueva	revisión de requisitos y prueba de interfaz	frontend 0.1.2–0.1.4
Exportador de PostgreSQL sin permiso para leer su credencial	OBS04	fsGroup

Conclusiones

- Los dos ambientes cumplen la matriz E01–E14 con evidencia reproducible, y prod puede reconstruirse desde cero en unos 11 minutos con un solo script.
- Las pruebas encontraron once defectos reales que una revisión estática no habría mostrado; cada uno tiene su evidencia, su corrección y, cuando aplica, una prueba de regresión.
- El HPA solo mejora el servicio cuando el cuello de botella está en el backend: con 0.1.1 subir de 2 a 6 réplicas no cambió el rendimiento porque PostgreSQL estaba saturado.
- La alta disponibilidad es parcial: la aplicación sobrevive a la pérdida de un Pod y al mantenimiento de un worker sin errores, pero cada base es un punto único de fallo y su volumen queda atado a un nodo.

Limitaciones

- Un solo host físico para ambos ambientes (y para otro clúster ajeno): sin tolerancia a la falla del host; los nodos de k3d comparten CPU y memoria aunque cada uno declare el total.

- Control plane único, una réplica por motor, almacenamiento local-path sin replicación.
- El p95 de 0.99 s cumple por poco: el siguiente límite es la CPU del backend y del host.
- Monitoreo dentro del mismo clúster que monitorea.
- Firewall de host documentado pero no activado; sin auditoría de la API de Kubernetes.
- Imagen base del backend sin digest; HIGH residuales en imágenes de terceros.
- Dos suspensiones del equipo afectaron pruebas largas (una invalidada, otra con un hueco de 4 h entre fases).

Referencias

- Kubernetes: https://kubernetes.io/docs/ (Workloads, Services, Ingress, NetworkPolicy, Pod Security Standards, RBAC, HPA, PodDisruptionBudget, CertificateSigningRequest)
- k3s: https://docs.k3s.io/ — k3d: https://k3d.io/
- Traefik: https://doc.traefik.io/traefik/
- Laravel 12: https://laravel.com/docs/12.x — Sanctum
- PostgreSQL: https://www.postgresql.org/docs/ — MongoDB: https://www.mongodb.com/docs/manual/
- kube-prometheus-stack: https://github.com/prometheus-community/helm-charts — Prometheus Operator CRD
- Grafana Loki: https://grafana.com/docs/loki/ — Grafana Alloy: https://grafana.com/docs/alloy/
- k6: https://grafana.com/docs/k6/ — Playwright: https://playwright.dev/
- Trivy: https://trivy.dev/ — kubeconform: https://github.com/yannh/kubeconform
- Patrón outbox: https://microservices.io/patterns/data/transactional-outbox.html
- Práctica individual Kubernetes: desarrollo y producción (documento del curso, no incluido en el repositorio)